

Logistic Regression in Python using sklearn

Beginner-friendly notes covering `LogisticRegression()`, `fit()`, `predict()`, `predict_proba()`, train-test split, confusion matrix, accuracy, precision, recall, and F1-score.

1. Import Logistic Regression

```
from sklearn.linear_model import LogisticRegression
```

LogisticRegression is the scikit-learn class used to create a Logistic Regression classification model.

2. Create the Model

```
model = LogisticRegression()
```

This creates the model, but the model has not learned from data yet.

3. Prepare X and y

Suppose we want to predict whether a student passes based on study hours.

```
X = [[1], [2], [3], [4], [5], [6]]
y = [0, 0, 0, 1, 1, 1]
```

X = Features / Independent Variable

y = Target / Dependent Variable

4. Train the Model with fit()

```
model.fit(X, y)
```

fit() trains the model. It looks at X and the correct answers y and learns the relationship between them, including model coefficients and intercept.

Remember: **fit() = learn from the training data.**

5. Make Predictions with predict()

```
model.predict([[7]])
```

The model returns a class, such as **array([1])**. Here, 1 could represent Pass and 0 could represent Fail.

6. predict() vs predict_proba()

Function	Returns	Example
<code>predict()</code>	Final class	<code>array([1])</code>
<code>predict_proba()</code>	Probability of each class	<code>[[0.20, 0.80]]</code>

```
model.predict_proba([[5]])
```

For output **[[0.20, 0.80]]**: Class 0 probability = 20%, Class 1 probability = 80%.

predict() → What class?

predict_proba() → How probable is each class?

7. Train-Test Split

In real Machine Learning, we should evaluate the model on data it did not use for training.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

test_size=0.2 means approximately 80% training data and 20% testing data. **random_state=42** makes the split reproducible.

8. Complete Basic Logistic Regression Code

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)
```

9. Confusion Matrix

A confusion matrix shows how many predictions were correct and incorrect.

	Actual 0	Actual 1
Predicted 0	TN	FN
Predicted 1	FP	TP

TP (True Positive): Actual 1, predicted 1.

TN (True Negative): Actual 0, predicted 0.

FP (False Positive): Actual 0, predicted 1. Type I error.

FN (False Negative): Actual 1, predicted 0. Type II error.

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

10. Accuracy

Question answered: Out of all predictions, how many were correct?

Accuracy = (TP + TN) / (TP + TN + FP + FN)

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print(accuracy)
```

11. Precision

Question answered: Of all cases predicted as positive, how many were actually positive?

Precision = $TP / (TP + FP)$

```
from sklearn.metrics import precision_score
precision = precision_score(y_test, y_pred)
print(precision)
```

Easy memory: Precision = When the model says YES, how often is it correct?

12. Recall

Question answered: Of all actual positive cases, how many did the model correctly identify?

Recall = $TP / (TP + FN)$

```
from sklearn.metrics import recall_score
recall = recall_score(y_test, y_pred)
print(recall)
```

Easy memory: Recall = Of all actual YES cases, how many did the model find?

13. F1-Score

F1-score combines Precision and Recall into a single measure.

$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$

```
from sklearn.metrics import f1_score
f1 = f1_score(y_test, y_pred)
print(f1)
```

F1-score ranges from 0 to 1, with 1 representing a perfect score.

14. Example: TP = 80, TN = 90, FP = 10, FN = 20

Metric	Calculation	Result
Accuracy	$(80+90)/200$	85%
Precision	$80/(80+10)$	88.89%
Recall	$80/(80+20)$	80%
F1-score	$2 \times (0.8889 \times 0.8) / (0.8889 + 0.8)$	84.21%

15. Complete Evaluation Code

```
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

print('Confusion Matrix:')
print(confusion_matrix(y_test, y_pred))

print('Accuracy:', accuracy_score(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred))
print('Recall:', recall_score(y_test, y_pred))
print('F1 Score:', f1_score(y_test, y_pred))
```

16. Quick Cheat Sheet

Function	Meaning
LogisticRegression()	Creates Logistic Regression model
fit(X, y)	Trains / learns from data
predict(X)	Returns class predictions
predict_proba(X)	Returns class probabilities
confusion_matrix()	Shows TN, FP, FN, TP
accuracy_score()	Overall correctness
precision_score()	Correctness of positive predictions
recall_score()	How many actual positives were found
f1_score()	Balance between precision and recall

17. Complete ML Flow

Data → **X and y** → **Train-Test Split** → **Create Model** → **fit()** → **predict()** / **predict_proba()** → **Evaluate** → **Accuracy / Precision / Recall / F1**

Most important distinction:

predict() → What class?

predict_proba() → How probable is each class?

Study tip: First understand TP, TN, FP, and FN. Accuracy, precision, recall, and F1-score are different ways of using those same four values.